



Desarrollo Web Profesional

José Miguel Carrera Pacheco



Emmanuel Castro Salvador

8.-A

**Investigación: Arquitectura de
autenticación y control de roles**

08/03/2026

Introducción

En el desarrollo de aplicaciones modernas, uno de los aspectos más críticos es la seguridad. Los sistemas actuales manejan grandes cantidades de información sensible, incluyendo datos personales, información financiera y acceso a servicios digitales. Por esta razón, es fundamental diseñar correctamente los mecanismos de autenticación y control de acceso desde las primeras etapas del desarrollo.

Una arquitectura de autenticación bien diseñada permite verificar la identidad de los usuarios y controlar qué acciones pueden realizar dentro de un sistema. Esto se logra mediante la implementación de mecanismos como sesiones, tokens, protocolos de autorización y sistemas de roles y permisos.

El diseño de estos mecanismos no debe realizarse de forma improvisada durante la programación, sino que debe ser planificado previamente como parte de la arquitectura del sistema. Un error en el diseño de la autenticación puede provocar vulnerabilidades críticas que comprometan la seguridad del sistema completo.

Autenticación y autorización

¿Qué es la autenticación?

La autenticación es el proceso mediante el cual un sistema verifica la identidad de un usuario o entidad que intenta acceder a un recurso. En otras palabras, la autenticación responde a la pregunta: *¿Quién eres?* Este proceso normalmente se realiza mediante credenciales que el usuario proporciona al sistema. Entre los métodos de autenticación más comunes se encuentran:

- Usuario y contraseña
- Autenticación mediante tokens
- Autenticación biométrica
- Autenticación multifactor (MFA)

Cuando un usuario introduce su nombre de usuario y contraseña en un sistema, el servidor verifica si esas credenciales coinciden con la información almacenada en la base de datos. Si la verificación es correcta, el sistema considera que el usuario está autenticado.

¿Qué es la autorización?

La autorización es el proceso que determina qué acciones puede realizar un usuario una vez que ha sido autenticado. Mientras que la autenticación responde a la pregunta: *¿Quién eres?* La autorización responde a: *¿Qué puedes hacer dentro del sistema?*

Por ejemplo:

- Un usuario normal puede ver su información personal.
- Un administrador puede modificar información de otros usuarios.
- Un moderador puede eliminar contenido.

La autorización se implementa generalmente mediante roles y permisos, los cuales permiten definir diferentes niveles de acceso dentro del sistema.

Sistemas de autenticación modernos

En el desarrollo web moderno existen diferentes métodos para manejar la autenticación de los usuarios. Entre los más utilizados se encuentran:

- Sesiones
- Tokens JWT
- OAuth

Cada uno de estos enfoques tiene ventajas y desventajas dependiendo del tipo de aplicación que se esté desarrollando.

JWT (JSON Web Token)

¿Qué es un JWT?

Un JSON Web Token (JWT) es un estándar abierto que permite transmitir información segura entre dos partes en forma de objeto JSON. Este token contiene información codificada que el servidor puede verificar mediante una firma digital. Un JWT generalmente contiene tres partes:

1. Header – Información sobre el tipo de token y algoritmo de firma.
2. Payload – Datos del usuario o claims.
3. Signature – Firma digital que garantiza la integridad del token.

Funcionamiento de JWT

El flujo básico de autenticación con JWT es el siguiente:

1. El usuario envía sus credenciales al servidor.
2. El servidor valida las credenciales.
3. Si son correctas, el servidor genera un token JWT.
4. El cliente almacena el token.
5. En cada petición posterior, el cliente envía el token al servidor.
6. El servidor valida el token antes de permitir el acceso.

Este método es muy utilizado en:

- APIs REST
- aplicaciones móviles
- microservicios

Sesiones

¿Qué son las sesiones?

Las sesiones son un mecanismo tradicional de autenticación en aplicaciones web. Cuando un usuario inicia sesión:

1. El servidor crea una sesión.
2. Se genera un identificador único de sesión.
3. Este identificador se guarda en una cookie del navegador.
4. En cada petición el navegador envía el ID de sesión.

El servidor utiliza ese identificador para reconocer al usuario autenticado.

OAuth

¿Qué es OAuth?

OAuth es un protocolo de autorización que permite a una aplicación acceder a recursos de otra aplicación en nombre de un usuario sin compartir sus credenciales.

Un ejemplo común es cuando un sitio web permite iniciar sesión usando:

- Google
- Facebook
- GitHub

En este caso, el sistema externo se encarga de autenticar al usuario.

Funcionamiento de OAuth

El flujo general de OAuth es el siguiente:

1. El usuario solicita iniciar sesión mediante un proveedor externo.
2. El proveedor autentica al usuario.
3. El proveedor genera un token de acceso.
4. La aplicación utiliza ese token para obtener información del usuario.

Este sistema es muy utilizado en aplicaciones modernas porque permite integrar servicios de autenticación externos de forma segura.

¿Cuándo conviene usar tokens y cuándo sesiones?

La elección entre tokens y sesiones depende del tipo de aplicación.

Uso de sesiones

Las sesiones son más apropiadas para:

- aplicaciones web tradicionales
- sistemas con arquitectura monolítica
- aplicaciones donde el servidor controla completamente el estado

Uso de tokens

Los tokens JWT son más apropiados para:

- APIs REST
- microservicios
- aplicaciones móviles
- arquitecturas distribuidas

Riesgos de seguridad en un mal diseño de autenticación

Un diseño incorrecto de autenticación puede provocar múltiples vulnerabilidades de seguridad. Entre los riesgos más comunes se encuentran

Diseño del flujo de autenticación

Un sistema seguro debe definir claramente el flujo completo de autenticación. Este flujo incluye:

1. Registro de usuario
2. Inicio de sesión
3. Generación de token o sesión
4. Verificación de permisos
5. Acceso a recursos protegidos
6. Cierre de sesión

Este diseño permite establecer controles de seguridad claros antes de implementar el sistema.

Importancia de diseñar la seguridad antes de programar

El diseño de la arquitectura de seguridad debe realizarse antes de comenzar el desarrollo del software. Esto permite anticipar riesgos y definir mecanismos adecuados de protección.

Diseñar la seguridad desde el inicio evita problemas graves en etapas posteriores del desarrollo y facilita la implementación de controles de acceso adecuados.

Un sistema que no considera la seguridad desde su arquitectura inicial puede ser vulnerable a ataques que comprometan la integridad de la información y la confianza de los usuarios.

Conclusión

La autenticación y el control de acceso son elementos fundamentales dentro de la arquitectura de cualquier sistema moderno. Implementar correctamente estos mecanismos permite garantizar que solo los usuarios autorizados puedan acceder a los recursos del sistema.

Durante esta investigación se analizaron conceptos clave como autenticación, autorización, tokens JWT, sesiones y OAuth, así como los riesgos asociados a un diseño deficiente de seguridad.

También se exploró el uso de roles y permisos para controlar el acceso a diferentes funcionalidades dentro de una aplicación.

Diseñar correctamente la arquitectura de autenticación desde el inicio del desarrollo permite prevenir vulnerabilidades críticas y mejorar la seguridad general del sistema.

Por esta razón, el diseño de la seguridad debe considerarse una parte esencial del proceso de desarrollo de software.

Referencias (formato APA 7)

Fielding, R. T., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Authentication. Internet Engineering Task Force.

<https://datatracker.ietf.org/doc/html/rfc7235>

Hardt, D. (2012). The OAuth 2.0 Authorization Framework. Internet Engineering Task Force.

<https://datatracker.ietf.org/doc/html/rfc6749>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). Internet Engineering Task Force.

<https://datatracker.ietf.org/doc/html/rfc7519>

OWASP Foundation. (2021). OWASP Top 10: The Ten Most Critical Web Application Security Risks.

<https://owasp.org/www-project-top-ten/>

Rapid7. (2024). Metasploit Framework documentation.

<https://docs.rapid7.com/metasploit/>

Stallings, W., & Brown, L. (2018). Computer security: Principles and practice (4th ed.). Pearson.

NIST. (2020). Digital identity guidelines. National Institute of Standards and Technology.

<https://pages.nist.gov/800-63-3/>